The Request

You are a Senior Embedded Engineer at **DeepSea Developments**, an engineering consultancy. We have just landed a contract with a new customer: a major national commercial EV charging network.

The customer's Lead Systems Engineer has sent our team the following urgent request:

> *Our Level 3 DC Fast Chargers, which use headless Raspberry Pi Zero Ws as their local communication gateways, are dropping data and occasionally locking up during peak charging sessions. We suspect CPU saturation or thermal throttling. The Pi is mounted inside the main sealed enclosure, relatively close to the high-voltage power electronics, meaning the processor's temperature is directly affected by the heat from the charging modules.*
>
> *Because these enclosures are weather-sealed and contain lethal voltages, field technicians cannot open them while the station is energized. The only safe way to interact with the nodes is via SSH over the station's local maintenance Wi-Fi link.*
>
> *We recently added heavy smart-grid features to the gateway, and now it becomes unstable. We need a custom diagnostic pipeline to monitor the Pi's internal metrics (CPU, RAM, Core Temperature). We need the fastest possible sample rate—ideally around **2,000 Hz (2kHz)**—to catch tight peaks in resource usage or sudden thermal spikes the exact millisecond a vehicle initiates a high-power charge.*

Your task at DeepSea Developments is to design and implement this system for the charging network. You must treat this as a production-grade component, balancing their demanding specifications with the physical realities of the hardware.

This is not just a coding test; it is an architecture and system design evaluation.

Technical Requirements

While the customer's request focuses on the end result, the DeepSea engineering team has defined the following strict technical constraints for your implementation:

- **Language:** Pure C (C99 or C11).
- **Libraries:** GLib (GDBus).
- **IPC Framework:** You must use the **D-Bus System Bus** to broadcast the telemetry data. This is necessary because the existing gateway implementation already uses D-Bus, and the telemetry feed will eventually be integrated so the gateway can natively log these metrics.
- **Resource Efficiency:** Your solution must **not** consume all the CPU or memory resources. Several other heavy services (like payment processing and grid communication) run on this gateway, so footprint and efficiency are paramount.
- **Latency Profiling:** The system must include a mathematical way to measure, calculate, and display the actual system latency to prove the real-time capabilities to the customer.

Core Components

You must write two distinct POSIX C applications that run concurrently:

Component A: The Telemetry Provider (Daemon)

- **Data Ingestion:** Must read real hardware metrics:
  - CPU Utilization (%)
  - RAM Utilization (%)
  - Core SoC Temperature (°C)
- **Frequency:** The system must account for the customer's 2,000 Hz sampling and transmission requirement.
- **IPC Protocol:** Must broadcast the payload using the D-Bus System Bus.

Component B: The Diagnostic Dashboard (Client)

- **Data Consumption:** Must subscribe to the Provider's D-Bus stream asynchronously.
- **Visualization:** Must render a real-time, graphical, in-place terminal dashboard displaying the metrics (Do not use **ncurses** or external UI frameworks).
- **Performance Matrix:** The dashboard must calculate and display real-time profiling metrics to satisfy the latency requirement:
  - Current Latency
  - Maximum Latency observed
  - Total Dropped/Missed Messages

How to Start

The system must run entirely on a headless **Raspberry Pi Zero W**. To simulate the remote, sealed nature of the EV chargers, you will access a provided physical test device remotely via **Raspberry Pi Connect**. To receive your device access:

- **Step 1: Generate an Auth Key.** Please create a free account at connect.raspberrypi.com.
- **Step 2: Share the Key.** Go to your **Settings**, click **Create Auth Key**, and send that key to our team. We will use this key to link the test hardware securely to your account, granting you direct SSH access.
- **Step 3: Begin Development and Testing.** Once we confirm your key is linked, you can SSH into the Raspberry Pi Zero W via the Raspberry Pi Connect portal. From there, you can clone your code repository, compile your binaries, and begin testing your telemetry broker directly on the target hardware.

Deadline and Submission

You have exactly **1 day (24 hours)** from the moment you receive this challenge and confirm hardware access to complete your implementation.

To submit your work, please share a link to a **Public GitHub or GitLab Repository**. Your repository must include your complete implementation, build files, and a comprehensive README.md that documents your architectural decisions and provides run instructions.